Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

A reading summary of Lewis et al. (2020), arXiv:2005.11401. Prepared as a sample document for the OAF Build Fellowship RAG project.

Abstract

Lewis and colleagues at Facebook AI Research introduce Retrieval-Augmented Generation (RAG), a family of models that combine a parametric sequence-to-sequence generator with a non-parametric memory implemented as a dense vector index of Wikipedia. The combined model retrieves a small set of supporting passages for each input, conditions generation on those passages, and is trained end-to-end. On a battery of knowledge-intensive tasks — open-domain question answering, abstractive question answering, fact verification, and Jeopardy-style question generation — RAG sets new state-of-the-art results, and beats both pure parametric baselines (BART, T5) and earlier retrieve-then-extract systems on most benchmarks.

Background and motivation

Large pre-trained language models like BART, T5 and GPT-3 store an impressive amount of world knowledge implicitly in their parameters. However this implicit knowledge is hard to inspect, hard to update, and limited by training cut-off and parameter count. When asked a factual question whose answer the model does not reliably 'know,' it tends to hallucinate plausible-sounding but incorrect responses.

A complementary line of work tackles this with retrieval-based models: REALM (Guu et al.) and DPR (Karpukhin et al.) showed that augmenting a model with a learnable retriever over a Wikipedia index improves open-domain QA accuracy. But those systems are extractive: they choose a span from a retrieved passage as the answer. They cannot generate a freeform answer that synthesises across documents.

RAG's contribution is to wire a learnable retriever into a generative seq2seq model, and to train the whole thing end-to-end, so that the generator can produce answers that are both grounded in retrieved evidence and stylistically free-form.

Architecture

The architecture has three components that act in sequence on every input:

1. Query encoder. A BERT-base model that maps the input sequence x to a single dense vector q(x). This is the half of the retriever that is fine-tuned during RAG training.

2. Document index. A pre-built FAISS index over roughly 21 million Wikipedia chunks (each about 100 words). Every chunk is encoded once, in advance, by a frozen BERT-base passage encoder. At

query time, an inner-product search returns the top-K chunks (typically K=5 or K=10).

3. Generator. A BART-large seq2seq model that takes the concatenation of the original input and a retrieved chunk, and produces an output sequence token by token.

The paper proposes two variants. **RAG-Sequence** uses the same retrieved document for the entire output sequence: it marginalises over documents at the sequence level. **RAG-Token** can attend to a different retrieved document at each generation step: it marginalises at the token level. RAG-Token is more flexible for outputs that draw on multiple sources; RAG-Sequence is simpler and performs better on tasks where the answer comes from one passage.

Training

RAG is trained end-to-end by maximising the marginal log-likelihood of the target output, where the marginalisation is over the top-K retrieved documents. Crucially, the gradient flows back through the query encoder, so the retriever learns to fetch documents that the generator can actually use. The passage encoder and the document index are frozen during this training — re-encoding the entire 21M-chunk index after every step would be infeasible.

The generator is initialised from BART-large pre-trained on the standard denoising objective. The query encoder is initialised from the DPR question encoder, which has itself been pre-trained on Natural Questions. Training uses Adam, learning rate 3e-5, batch size 64 across 8 GPUs, for between one and ten epochs depending on the downstream task.

An important detail: because the document index is frozen, the retriever can only learn to project the query into the existing document embedding space. It cannot learn to embed documents better. The paper notes this as a known limitation and an avenue for future work.

Results

On four open-domain question answering benchmarks — Natural Questions, TriviaQA, WebQuestions, and CuratedTrec — RAG sets new state-of-the-art results for generative models, beating T5-11B despite having an order of magnitude fewer parameters. On Natural Questions, RAG-Sequence achieves 44.5 exact-match, compared to 36.6 for T5-11B and 41.5 for the strongest extractive baseline at the time.

On the MS-MARCO abstractive question answering task, RAG outperforms BART by a large margin and generates answers that human raters consistently prefer for factuality, even though they are sometimes shorter or less fluent. On the FEVER fact verification benchmark, RAG matches state-of-the-art with substantially less task-specific engineering.

Qualitative analysis shows that RAG produces answers grounded in the retrieved evidence: the generated text tends to contain phrases that appear in the retrieved chunks, and ablating retrieval (running the generator alone) substantially degrades factual accuracy.

Why it matters

RAG made three contributions that have shaped how production AI systems are built today.

First, it formalised the now-standard pattern of dense retrieval + generation. The pattern — embed the query, fetch the top-K, condition generation on them — is the architecture running under most enterprise document Q&A products in 2025.

Second, it showed that adding a non-parametric memory is a cost-effective alternative to scaling parameters: an 11-billion-parameter model is beaten by a 400-million-parameter generator with access to a Wikipedia index. This trade-off — smaller model, external memory, lower cost — is the

reason RAG has displaced fine-tuning for many knowledge-update use cases.

Third, it made the knowledge source inspectable and updatable. Want the model to know about events after its training cut-off? Add those documents to the index. Want to remove a source? Delete it. This separation of model from knowledge is the property that has made RAG, rather than larger parametric models alone, the dominant architectural pattern for grounded AI applications.

The teaching codebase you are reading right now is a direct descendant of this paper: replace BART with Claude or Gemini, replace FAISS with ChromaDB, and the loop — embed query, retrieve passages, condition the generator — is the same.